



SafeWay

Rapport de Projet Génie Logiciel :

Matière : Génie Logiciel

Nom de la tutrice : Madame Sirine Hawari

Sujet : Développer une application de simulation de déplacement d'agents dans un bâtiment lors d'une évacuation dans un cas d'urgence

Thématique n°3 : Déplacement d'agents dans un graphe

Filière : ING1 Génie Informatique

Réalisé par le Groupe GI05-B :

- MEHANNI Rayane
- MURUGAN Alexis
- OZSOLAK Atahan
- POLAT-DEMIRKAYA Erwan
- SAOULI Rémi

Établissement : CY Tech, avenue du Parc, 95000, Cergy

Année universitaire : 2025/2026

Sommaire

Introduction.....	3
Technologies utilisées.....	4
Planning.....	6
Semaine du 18/05.....	6
Semaine du 25/05.....	7
Semaine du 01/06.....	8
Semaine du 08/06.....	8
Architecture finale de l'application.....	10
Diagrammes UML.....	12
Diagramme de classes.....	12
Couche Modèle (model).....	13
Couche Controlleur (controller).....	15
Couche Vue (view).....	15
Fonctionnement de la simulation.....	18
Difficultés.....	20
Conclusion.....	22
Annexes.....	23

Introduction

Lors d'évacuations en cas d'urgence, surtout dans les grands bâtiments dans lesquels on peut trouver plusieurs occupants, plusieurs victimes succombent à cause des bousculades qui peuvent être mortelles lors des déplacements, ainsi qu'à la saturation des portes, qui donne lieu à des comportements humains empirant la situation. C'est dans cette perspective que notre solution a vu le jour : SafeWay, un dispositif permettant de guider les occupants d'un bâtiment vers les sorties les plus optimisées tout en évitant la saturation des portes.

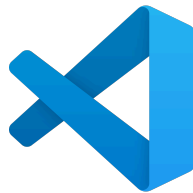
Notre projet en génie logiciel vient en soutien à notre solution afin de pouvoir simuler notre dispositif et ainsi observer le facteur dont dépend le plus ce dernier : le facteur humain. Le projet nous permet de faire une analyse sur différents échantillons avec plusieurs visuels, de sorte à observer les déplacements en temps réel des occupants et leurs interactions entre eux et avec les destinations.

Ce projet a été réalisé par notre groupe GI5-B et constitue la suite de notre projet en éthique et en design.

Technologies utilisées

Pour développer notre projet en génie logiciel, qui consiste en une simulation de déplacement d'agents dans un graphe, nous nous sommes appuyés sur les technologies suivantes :

- **VS Code** : Un éditeur de code source gratuit et open-source. C'est un outil très puissant et polyvalent qui permet de coder dans plusieurs langages différents.



- **Git** : Un logiciel de gestion de versions décentralisé. Il est gratuit et permet de gérer les ajouts et changements apportés au code source de manière tracée.



- **GitHub** : Une plateforme d'hébergement publique et gratuite qui permet de stocker, se partager et travailler sur des projets en groupe.



- **Java** : Un langage de programmation de haut niveau orienté objet permettant entre autres de développer des logiciels.



- **JavaFx** : Un framework qui permet de créer des applications Java dotées d'une interface utilisateur moderne hautement portable.



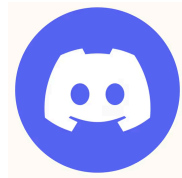
- **Maven** : Un outil de gestion et d'automatisation de production des projets logiciels Java, utilisé pour automatiser l'intégration continue lors d'un développement de logiciel.



- **Jira** : Un système de gestion de projet permettant à des équipes de travailler ensemble.



- **Discord** : Une plateforme de communication qui permet aux utilisateurs d'interagir par texte, voix et vidéo.



Planning

Sans compter le travail sur notre projet côté éthique et design, nous avons travaillé pendant quatre semaines sur le volet technique en équipe, accompagnés par notre tutrice, Madame **Sirine Hawari**. Notre travail sur cette période s'est déroulé de la manière suivante :

Semaine du 18/05

Dès que nous avons eu accès au cahier des charges et à la liste de nos tuteurs, nous nous sommes élancés pour les lire et commencer à visualiser le projet. Après cela, nous avons contacté notre tutrice attitrée pour organiser une première réunion afin de discuter de la marche à suivre ainsi que des questions auxquelles nous avions réfléchi lors de la lecture du cahier des charges. La réunion s'est déroulée impeccablement, notamment parce que nous étions préparés et que la tutrice nous a directement donné les prochaines étapes à suivre afin de pouvoir compléter le projet et d'avoir un rendu adéquat. À la fin de la réunion, la tutrice nous a aidés à nous organiser avant de nous lancer dans le projet en mettant en valeur plusieurs outils importants afin de pouvoir compléter un MVP démontrable pour la prochaine réunion.

Pour ce qui est du partage des tâches et du suivi de ces dernières, nous avons mis en commun un projet Jira, qui est un logiciel permettant de garder une trace des différentes tâches au cours d'un projet, de leur état et des personnes auxquelles elles sont assignées, afin de savoir en temps réel qui travaille sur quoi et ce qui doit être fait. Cette façon d'organiser les tâches nous a permis d'aller plus rapidement en évitant les complications liées à l'identification des fonctionnalités à développer, ce qui nous a permis d'avancer plus efficacement. Le fait d'avoir créé un dépôt GitHub nous a aussi facilité la tâche, nous permettant de partager notre code tout en gardant un historique des versions. De cette façon, chacun peut travailler sur les parties qui le concernent sans pour autant retarder les autres.

Pour ce qui est de l'organisation au sein de l'équipe, nous nous sommes divisés les tâches en fonction des affinités et des envies de chacun. Rayane et Erwan ont décidé de s'occuper du backend, en créant les modèles qui contiennent les classes métier et la logique métier dont a besoin l'application en termes de nœuds, d'agents, d'arêtes et de simulation. Alexis, Atahan et Rémi, quant à eux, se sont occupés du côté frontend ainsi que du côté contrôleur pour la liaison entre la logique métier et l'interface graphique. Grâce à cela, nous avons pu nous répartir les tâches de façon homogène sur différents aspects du projet afin d'avancer en même temps et efficacement sous une architecture MVC (Model - View - Controller). Après cela, toute l'équipe a décidé de commencer à

travailler sur ses tâches après les avoir ajoutées sur Jira afin d'entamer l'implémentation du projet.

Semaine du 25/05

La semaine d'après s'est surtout consacrée au début de l'implémentation du MVP que nous devons montrer à la tutrice lors de la prochaine réunion. Rayane a commencé en créant l'architecture des fichiers pour que chacun puisse travailler sur des fichiers différents et a commencé à créer les classes de logique métier. Il a développé les éléments essentiels d'un graphe, notamment les nœuds et les arêtes, ainsi que les agents qui s'y trouvent. Puis, il a ajouté une classe de simulation ainsi qu'un déplacement simple des agents suivant le plus court chemin. Erwan l'a aidé du côté du backend sur certaines classes et les relations entre elles.

Pendant ce temps, Rémi avait commencé à développer ses connaissances en JavaFx afin de pouvoir afficher les nœuds sur une page. Pour ce qui est de leur affichage, nous avons décidé de les disposer de façon circulaire afin de permettre aux arêtes de rester visibles sans se chevaucher malgré un grand nombre de nœuds. Alexis, lui, travaillait sur l'affichage des arêtes entre deux nœuds avec des flèches dirigées afin d'ajouter ces fonctionnalités à la page de Rémi. Il s'est aussi occupé de l'aspect visuel pour que les pages aient une direction artistique plus proche des maquettes que nous avons réalisées en design. Atahan, quant à lui, avait créé les pages de détails des différents éléments, notamment les nœuds, les arêtes et les agents, afin de permettre, après un clic sur l'un d'eux, de visualiser ses informations et ses caractéristiques.

Après que chacun eut terminé sa partie, nous avons intégré les fonctionnalités de chacun afin d'obtenir une première version de test qui nous a permis de visualiser ce qui fonctionnait et ce qui présentait des bugs. Grâce à cela, toutes les fonctionnalités implémentées par la suite ont été rapides à tester. Nous avons clôturé cette première partie du projet avec un diagramme de classes mettant en valeur toutes les classes utilisées dans notre projet ainsi que les relations entre elles.

Puis le jour de la deuxième réunion avec notre tutrice est arrivé, et c'est à ce moment-là que nous lui avons présenté notre MVP. Elle nous a donné de nombreuses remarques sur les éléments qui devaient être améliorés ainsi que sur les fonctionnalités qui devaient être ajoutées. Nous avons donc pris note de tout cela et nous étions satisfaits d'être relativement avancés, avec une version MVP du projet disposant déjà d'une interface graphique fonctionnelle.

Semaine du 01/06

Ensuite, chacun s'est penché sur les fonctionnalités à ajouter qui le concernaient, ainsi que sur les bugs qui devaient être corrigés. Côté logique métier, plusieurs changements ont été effectués, notamment dans la gestion des lieux (nœuds ou arêtes) et leur façon de stocker les agents. Les déplacements de ces derniers ont également été modifiés de sorte à ce que chaque état d'un agent corresponde à une manière de déterminer sa prochaine destination. Une personne paniquée, par exemple, suivra la foule, alors qu'une personne calme privilégiera plutôt le chemin le plus court afin d'atteindre la sortie plus rapidement. Pour ce qui est du déplacement au sein d'une arête, la vitesse de l'agent est exponentiellement décroissante en fonction de la congestion de cette dernière. De cette façon, une arête saturée causera davantage de bousculades, rendant le déplacement des agents plus difficile. Un système de fichiers a aussi été implémenté, permettant de sauvegarder l'état actuel de la simulation dans un fichier binaire dont le nom correspond à celui donné à la simulation. Ces sauvegardes peuvent ensuite être restaurées comme autre manière de lancer une simulation, au lieu d'en créer une nouvelle.

Du côté de l'interface graphique, beaucoup d'indicateurs visuels ont été ajoutés afin que la simulation soit claire, notamment au niveau de la coloration des nœuds, des arêtes et de leur positionnement. Plusieurs autres pages ont été créées pour permettre la création de nouveaux éléments (nœuds, arêtes ou agents) avec des attributs spécifiques. Comme auparavant, nous avons fait en sorte que la simulation puisse être exécutée par étapes et que l'on puisse passer de l'une à l'autre par un simple clic. Nous avons conservé cette fonctionnalité et en avons ajouté une autre où les mouvements sont temporels, permettant ainsi de visualiser la simulation en fonction du temps. Grâce à cela, nous avons implémenté l'essentiel des fonctionnalités nécessaires pour lancer et visualiser une simulation de déplacement d'agents dans un graphe.

Après avoir montré cette version avancée à notre tutrice, elle nous a donné des indications concernant le rapport ainsi que les diagrammes que nous devions préparer, en plus de quelques modifications UI/UX à entreprendre.

Semaine du 08/06

La dernière semaine a surtout été consacrée aux dernières modifications nécessaires à notre projet. Du côté de la logique métier, de la documentation a été ajoutée afin de pouvoir générer une Javadoc, et une factorisation du code a été effectuée pour simplifier ce dernier. L'objectif était de rendre le code clair afin qu'il puisse être facilement compréhensible par n'importe quelle personne. Du côté de l'interface graphique, quelques modifications ont été apportées, principalement au niveau du style, afin de permettre une meilleure visualisation du graphe et de ses éléments.

Après cela, nous nous sommes mis à travailler sur les diagrammes de classes et de cas d'utilisation de notre projet, ainsi que sur le README et le rapport afin de le finaliser. Une fois ces éléments terminés, nous avons organisé une dernière réunion avec notre tutrice afin de vérifier que tout était conforme et que rien n'avait été oublié.

Architecture finale de l'application

L'application a été conçue en respectant strictement le patron d'architecture logicielle MVC (Modèle-Vue-Contrôleur). Ce choix garantit une séparation claire des responsabilités : les données et la logique métier sont totalement indépendantes de l'interface graphique. Cette modularité permet de faire évoluer l'application (par exemple, ajouter de nouveaux comportements d'agents) sans risquer de casser l'affichage.

Le Modèle (La logique métier et les données) :

Le package `com.app.model` représente le cœur de l'application. Il contient toute la logique de simulation, de pathfinding et de structure des données. Il est totalement agnostique vis-à-vis de JavaFX.

L'Environnement (Graph, Node, Edge) : La topologie de la simulation est modélisée sous forme de graphe orienté. Les Node (nœuds) représentent des points d'intérêt ou des sorties, dotés d'une capacité maximale (`maxAgents`). Les Edge (arêtes) relient ces nœuds et possèdent une distance physique qui impacte le temps de trajet.

Les Entités Mobiles (Agent) : Les agents sont dotés d'une autonomie décisionnelle. Leurs déplacements sont influencés par leur état psychologique (`AgentState` : `CALM`, `PANICKED`...) et leur comportement face aux autres (`AgentBehavior`). Ils s'appuient sur un algorithme de résolution de chemin (ex: `ShortestTimePath`) pour trouver la sortie la plus proche.

Le Moteur de Simulation (Simulation) : C'est le chef d'orchestre de la logique. À chaque itération de la boucle de temps, il met à jour la position des agents (`move()`) et gère les sorties du graphe (`clearExits()`).

La Persistance (`SaveLoadManager`) : Ce module gère la sérialisation et la désérialisation du graphe. Il permet de sauvegarder l'état complet d'une configuration pour la recharger ultérieurement depuis un fichier.

La Vue (L'interface graphique JavaFX) :

Le package `com.app.view` gère l'affichage et les interactions avec l'utilisateur. L'interface a été pensée pour offrir une expérience utilisateur (UX) fluide, inspirée des logiciels de conception professionnels.

Structure Globale en `BorderPane` : Chaque écran majeur utilise un layout spatial divisé en zones stratégiques :

Top (Barre de navigation) : Permet le retour à l'accueil ou au graphe, et l'affichage du titre de la page actuelle.

Left / Right (Boîtes à outils) : Hébergent les actions contextuelles (création de nœuds, édition, contrôle de la vitesse x_1 , x_2 ...) et la légende dynamique (gradient de congestion).

Center (Zone de travail) : Affiche le rendu principal. L'utilisation de StackPane garantit le centrage absolu du graphe (nodePane), indépendamment de la taille de la fenêtre.

Feedback Visuel : L'état de la simulation est retranscrit visuellement en temps réel grâce à l'interpolation de couleurs (ex: gradient du blanc fluide au rouge saturé), permettant à l'utilisateur d'identifier instantanément les goulots d'étranglement (congestion).

Le Contrôleur (Le chef d'orchestre) :

Le MainController (package com.app.controller) fait le pont entre la Vue et le Modèle.

Gestion de la Navigation : Il détient la référence de la scène principale (Stage / Scene) et orchestre les transitions entre les différentes pages (showHome(), showGraph(), showCreateNode(), etc.).

Injection de Dépendance : Lorsqu'une vue doit afficher des informations spécifiques (comme les détails d'un nœud cliqué), le contrôleur instancie la page correspondante en lui injectant les données du Modèle nécessaires.

Centralisation de l'État : Il maintient la référence vers l'instance unique de la Simulation en cours, s'assurant que toutes les vues travaillent sur les mêmes données.

La Boucle de Simulation (Game Loop) :

Le temps de la simulation est cadencé par une Timeline JavaFX située dans la vue principale (GraphView). Cette boucle asynchrone (définie par des KeyFrame régulières, ex: 0.1s) appelle de manière synchronisée :

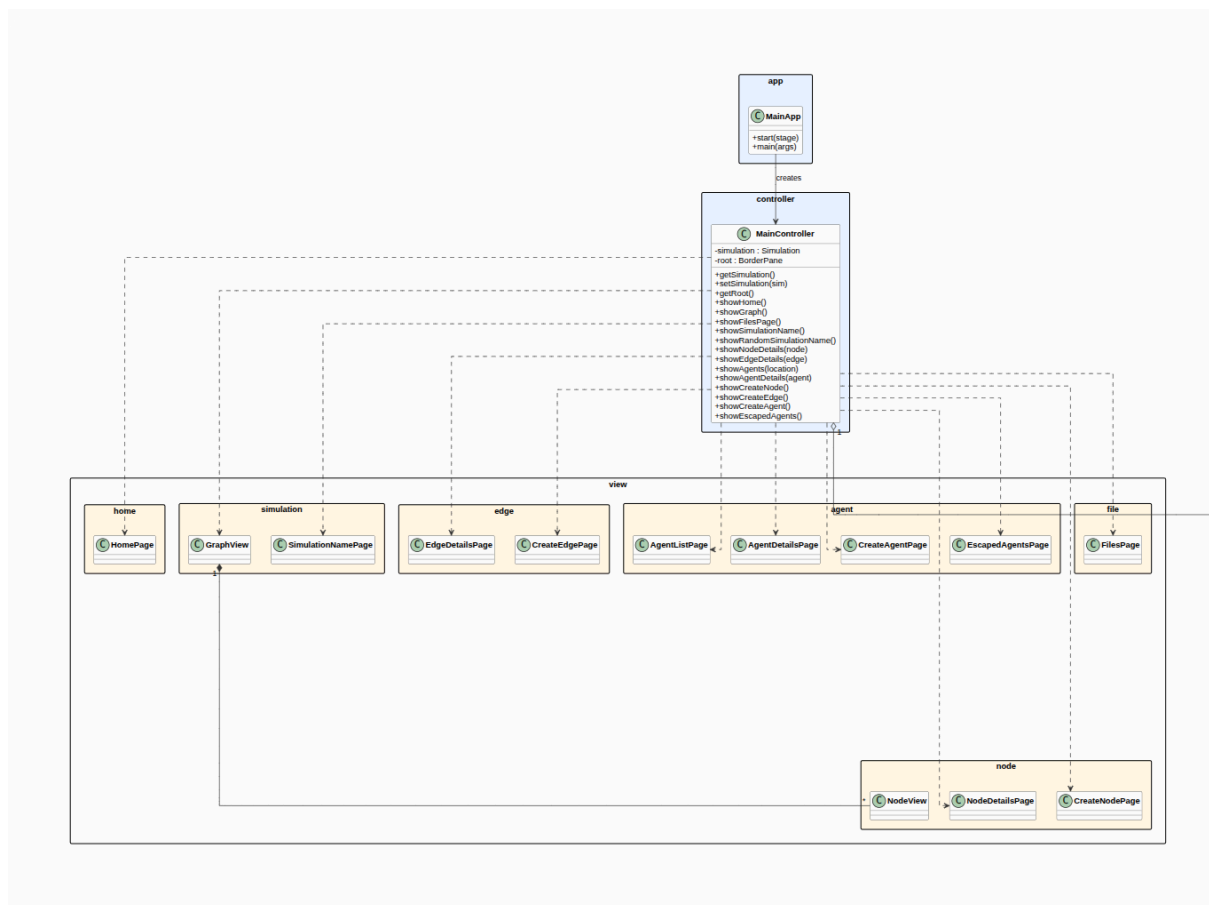
- La mise à jour mathématique du modèle (simulation.move()).
- Le nettoyage des agents arrivés à destination (simulation.clearExits()).
- Le rafraîchissement visuel du canevas (relayout()).

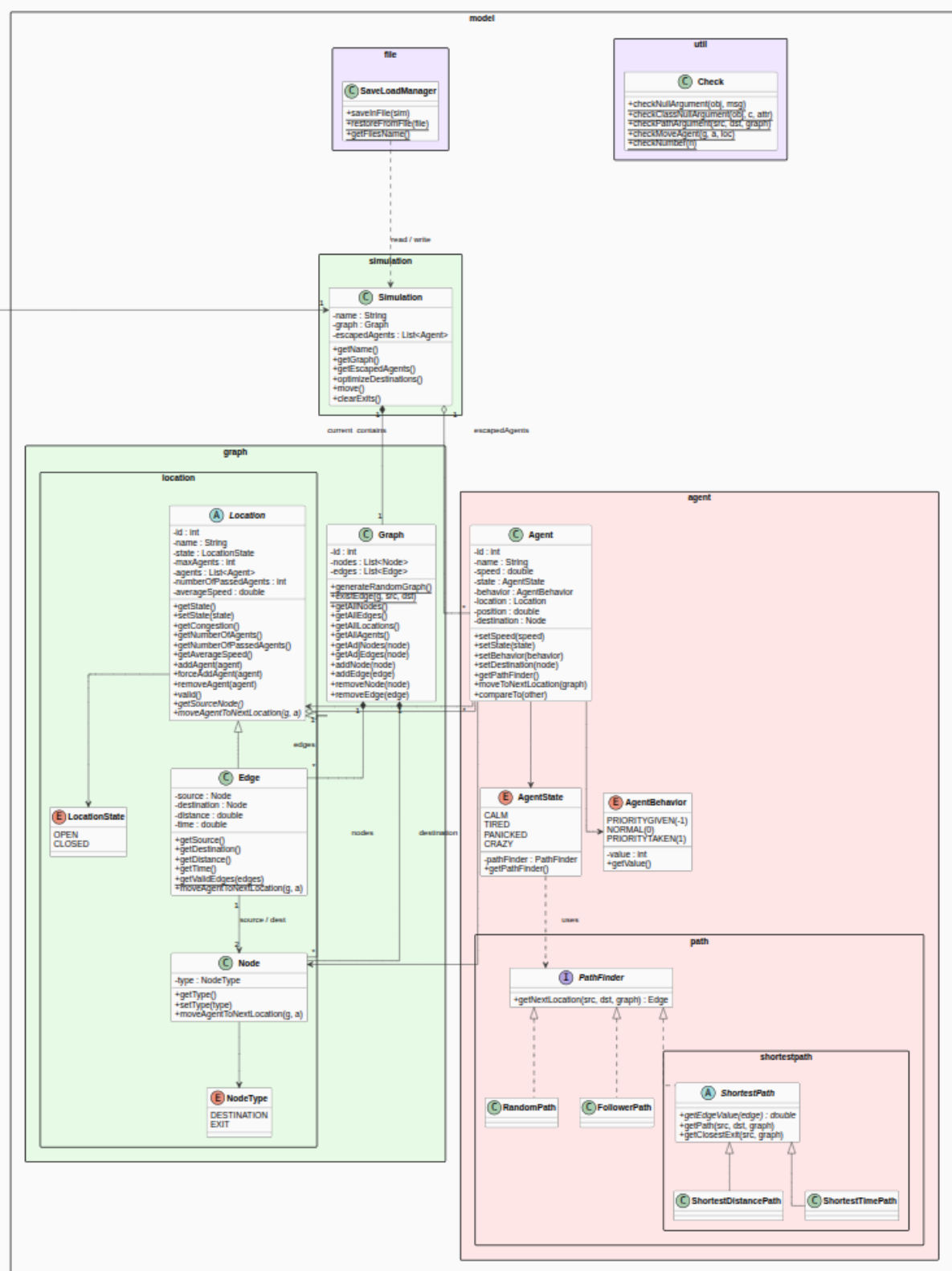
Ce couplage léger permet d'accélérer, ralentir ou mettre en pause la simulation (boutons Play/Stop/Next) sans bloquer le thread principal de l'interface utilisateur.

Diagrammes UML

À la fin de notre projet, nous avons réalisé plusieurs diagrammes UML afin de comprendre, d'une part, la structure du code et de l'architecture technique et, d'autre part, les différentes utilisations possibles de notre application, ainsi que les fonctionnalités qu'elle permet d'offrir et les résultats qu'elle peut produire.

Diagramme de classes





L'application suit une architecture MVC (Modèle – Vue – Contrôleur). Les classes principales se répartissent en trois grandes couches.

Couche Modèle (model)

- **Simulation** : Classe centrale du modèle. Elle représente une simulation en cours et possède un nom, un Graph et la liste des agents échappés (escapedAgents). Elle expose les opérations de progression de la simulation : move() (déplacement de tous les agents d'un pas), clearExits() (vidage des nœuds de type sortie) et optimizeDestinations() (assignation automatique des destinations).
- **Graphe** : Représente le graphe orienté pondéré : il contient la liste des nœuds (Node) et des arêtes (Edge). Il fournit les opérations d'ajout, de suppression et d'accès, ainsi que la méthode statique generateRandomGraph() qui produit un graphe aléatoire utilisé pour les nouvelles simulations.
- **Location (abstraite), Node, Edge** : Location factorise tout ce qui est commun à un lieu où se trouvent des agents : un identifiant, un nom, un état (LocationState : OPEN / CLOSED), une capacité maximale, la liste des agents présents et un indicateur de congestion (getCongestion()).

Deux classes héritent de Location :

- **Node** : un sommet du graphe, caractérisé par un NodeType (DESTINATION ou EXIT).
- **Edge** : une arête orientée entre deux nœuds, avec une distance et un temps de parcours.
- **Agent** : Représente un individu se déplaçant sur le graphe. Chaque agent possède une vitesse, un état (AgentState), un comportement (AgentBehavior), une position courante (Location + position entre 0 et 1), ainsi qu'une destination (un Node). La méthode moveToNextLocation() fait avancer l'agent d'un pas en fonction de son Pathfinder.
- **AgentState et AgentBehavior** sont deux énumérations qui définissent respectivement l'état psychologique et le comportement d'un agent. L'état peut être CALM, TIRED, PANICKED ou CRAZY, et chaque valeur est directement associée à un Pathfinder différent, ce qui constitue une application du pattern Strategy : modifier l'état d'un agent suffit à changer automatiquement sa façon de se déplacer. Le comportement, lui, peut être PRIORITYGIVEN, NORMAL ou PRIORITYTAKEN, et détermine l'ordre de passage des agents lorsqu'ils se retrouvent en situation de conflit.
- **PathFinder** est une interface qui définit la méthode getNextLocation(source, destination, graph) et qui est implémentée par plusieurs classes selon le type de déplacement souhaité. RandomPath permet un déplacement aléatoire, FollowerPath fait suivre la foule à l'agent, et ShortestPath est une classe abstraite dont héritent ShortestDistancePath, qui calcule le plus court chemin en distance,

et `ShortestTimePath`, qui calcule le plus court chemin en temps en tenant compte de la congestion.

Enfin, **SaveLoadManager** est la classe chargée de la sérialisation et de la désérialisation d'une Simulation vers et depuis un fichier .bin, ce qui permet de sauvegarder et de restaurer l'état complet d'une simulation.

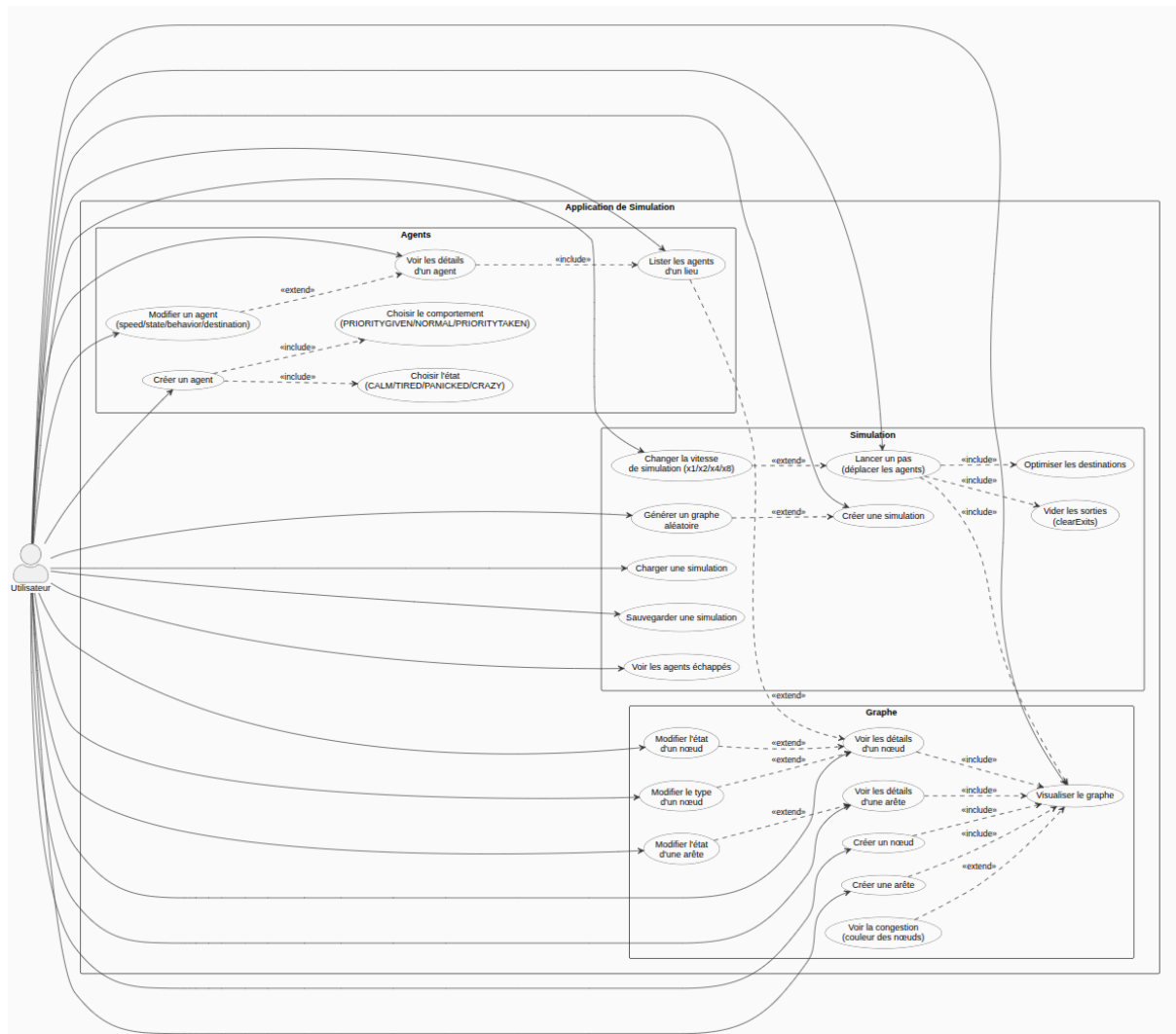
Couche Controlleur (controller)

L'application ne possède qu'un seul contrôleur : **MainController**. Il détient la simulation courante ainsi que la racine JavaFX (`BorderPane`), et toutes ses méthodes `showXxx()` — comme `showGraph`, `showNodeDetails`, `showAgents` ou `showEscapedAgents` — remplacent dynamiquement la page affichée au centre de l'interface. Il assure ainsi l'intégralité de la navigation entre les vues.

Couche Vue (view)

Elle regroupe l'ensemble des composants JavaFX dédiés à chaque écran de l'application. La **HomePage** correspond au menu d'accueil, tandis que la **GraphView** est la vue principale, qui affiche le graphe de manière interactive et propose une barre d'outils avec les boutons Play, Stop, Save et Agents escaped. Les pages **NodeDetailsPage**, **EdgeDetailsPage** et **AgentDetailsPage** permettent de consulter les détails d'un élément et de le modifier via un bouton Modify. **AgentListPage** et **EscapedAgentsPage** affichent les listes d'agents sous forme de ronds cliquables, et **CreateNodePage**, **CreateEdgePage** et **CreateAgentPage** sont des formulaires de création. Enfin, **FilesPage** et **SimulationNamePage** gèrent les fichiers de sauvegarde.

Diagramme de cas d'utilisation



Le diagramme de cas d'utilisation recense l'ensemble des interactions possibles entre l'utilisateur et l'application. Parmi celles-ci, les plus importantes — celles qui forment le cœur fonctionnel — sont les suivantes.

- **Créer une simulation** permet à l'utilisateur de démarrer depuis un graphe vide ou d'utiliser la fonctionnalité de génération aléatoire, qui produit automatiquement un graphe complet avec des nœuds, des arêtes et des agents.
- **Charger et sauvegarder une simulation** est une fonctionnalité centrale : l'utilisateur peut à tout moment sauvegarder l'état complet de la simulation dans un fichier et le recharger plus tard via le SaveLoadManager.

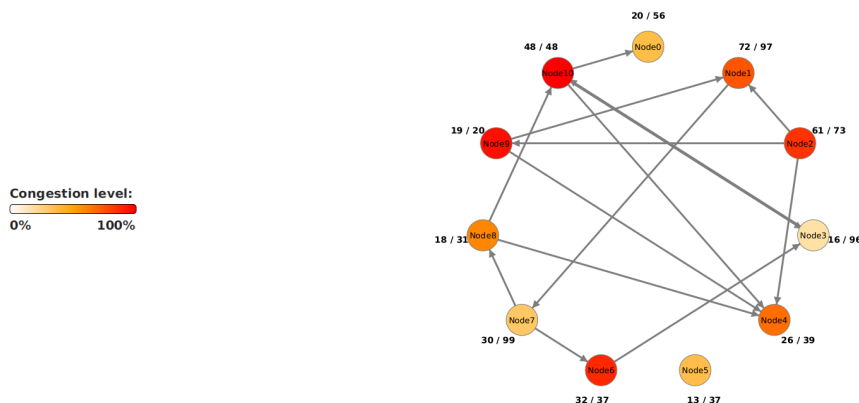
- **Visualiser le graphe** est le cas d'utilisation pivot de l'application, dont dépendent presque tous les autres — créer un nœud, créer une arête, consulter les détails via une relation <<include>>.
- **Lancer un pas de simulation** est l'action cœur du moteur : à chaque pas, l'application déplace les agents avec `move()`, optimise leurs destinations avec `optimizeDestinations()` et vide les nœuds de type sortie avec `clearExits()`.
- **Créer un nœud, une arête ou un agent** correspond à trois cas d'utilisation symétriques qui permettent à l'utilisateur de construire manuellement son graphe et sa population d'agents.
- **Voir les détails** d'un nœud, d'une arête ou d'un agent permet d'inspecter un élément précis. Ces cas sont prolongés par des cas de modification `changer l'état` d'un nœud ou d'une arête (`OPEN ↔ CLOSED`), ou modifier les attributs d'un agent (vitesse, état, comportement, destination) en une seule opération.
- **Lister et inspecter les agents d'un lieu** permet, depuis un nœud ou une arête, de consulter la liste des agents qui s'y trouvent et d'accéder à leurs détails individuels.
- **Voir les agents échappés** est un cas spécifique à la simulation : il permet d'afficher la liste des agents ayant atteint une sortie, ce qui donne une mesure concrète de l'efficacité de l'évacuation.
- **Voir la congestion** est un cas d'utilisation visuel : la couleur des nœuds évolue en temps réel selon leur taux de remplissage, passant du blanc à l'orange puis au rouge, ce qui permet une lecture immédiate de l'état de la simulation.

En synthèse, trois grandes familles de cas d'utilisation structurent l'application : la gestion de la simulation (créer, charger, sauvegarder, lancer un pas), l'édition du graphe (créer, visualiser et modifier les nœuds et les arêtes), et la gestion des agents (créer, lister, modifier et suivre les agents échappés).

Fonctionnement de la simulation

Lorsque l'application est lancée via le CMD, une application s'ouvre en grand écran (possibilité de mettre l'application en mode fenêtré en appuyant sur echap). L'application s'ouvre sur une page d'accueil présentant le logo de SafeWay et trois possibilités. L'utilisateur peut lancer une nouvelle simulation qui démarre à partir d'un graphe vide ; lancer une simulation aléatoire qui génère automatiquement un graphe rempli de nœuds, d'arêtes et d'agents ; ou consulter ses sauvegardes pour reprendre une simulation enregistrée. Dans les deux premiers cas, l'utilisateur doit donner un nom à sa simulation.

Une fois la simulation lancée, l'utilisateur arrive sur la vue principale : le graphe. Les nœuds sont représentés par des cercles colorés, les arêtes par des lignes munies d'une flèche indiquant leur sens, et les agents par de petits cercles orange numérotés. La couleur des noeuds change en temps réel selon leur niveau de congestion : un noeud qui se teinte du blanc vers l'orange et le rouge à mesure qu'il se remplit allant de 0% à 100%.

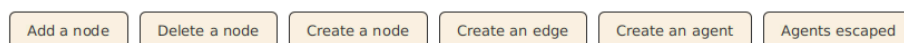


Un clic sur un nœud ouvre sa page de détails qui affiche son identifiant, son nom, son état, son type et son taux d'occupation ; un clic sur une arête ouvre de même sa page de détails avec sa source, sa destination et son état. Sur ces pages, un bouton « Modify » fait passer la page en mode édition et permet de modifier l'état de l'élément avant de l'enregistrer. L'utilisateur peut aussi consulter les agents présents sur le nœud/arête via « See agents », ou supprimer l'élément concerné. La page des agents d'un lieu affiche chaque agent sous forme de cercle cliquable ; en cliquant sur l'un d'eux, l'utilisateur accède à sa propre page de détails (nom, vitesse, état, comportement, position, destination), elle aussi modifiable et permettant de supprimer l'agent.

La barre d'outils propose la création d'un noeud (en précisant son état, son type et sa capacité maximale), d'une arête (en choisissant une source, une destination et une

distance) et d'un agent (en renseignant son nœud de départ, son nom, sa vitesse, son état, son comportement, sa tolérance à la congestion et éventuellement sa destination). On peut aussi créer un nœud, une arête ou un agent directement depuis la page de détails d'un nœud. Dans ce cas, ce nœud est déjà sélectionné comme source de l'arête ou comme point de départ de l'agent, ce qui fait gagner du temps. De la même façon, chaque élément peut être supprimé depuis sa page de détails.

Les formulaires vérifient ce que l'utilisateur saisit. Par exemple, une arête doit relier deux nœuds différents qui ne sont pas déjà reliés, et la distance comme la vitesse doivent être des nombres positifs. Si une saisie est incorrecte, un message d'erreur s'affiche et l'application continue de fonctionner normalement.



Une fois le graphe en place, l'utilisateur peut commencer la simulation. Le bouton « Next » fait avancer la simulation d'un pas : tous les agents se déplacent vers leur prochaine position selon leur destination et leur comportement, et le graphe se redessine pour refléter le nouvel état. L'utilisateur peut aussi lancer une simulation automatique avec les « Play » et « Stop ». L'utilisateur peut régler la vitesse de défilement grâce aux multiplicateurs (x1, x2, x4, x8). À chaque pas, les agents qui atteignent une sortie quittent le bâtiment. Ils sont alors ajoutés à la liste des agents échappés, que l'utilisateur peut consulter à tout moment avec le bouton « Agents escaped ».

Enfin, l'utilisateur peut sauvegarder la simulation dans un fichier portant son nom, pour la reprendre plus tard depuis l'écran des sauvegardes. Il peut aussi quitter la simulation et revenir à la page d'accueil. Toutes ces actions permettent de construire, modifier, lancer et analyser une évacuation, et d'observer l'effet du comportement humain sur les déplacements et la saturation des sortie

Difficultés

Lors de la réalisation de ce projet en génie logiciel, la plus grande difficulté que nous avons rencontrée était la gestion du temps. Entre les rattrapages, les emplois du temps de chacun et les procédures de mobilité internationale pour l'ING2, il était très difficile de regrouper à chaque fois tout le groupe pour une réunion d'équipe. Nous avons quand même pu pallier cela en multipliant les réunions et en communiquant régulièrement.

Le temps a également été un défi, donc nous n'avons pas eu l'occasion d'implémenter plusieurs fonctionnalités bonus auxquelles nous avons réfléchi. Au vu du temps qu'il nous restait, nous avons donc décidé d'implémenter uniquement les fonctionnalités demandées, tout en faisant attention à ce que l'application fonctionne parfaitement, sans bugs ni problèmes lors de l'exécution.

Le dernier problème que nous avons rencontré est survenu au début du projet concernant la partie interface graphique avec JavaFx. Comme nous n'avions pas forcément d'idée de la manière dont sont gérés les projets JavaFx, nous avons été bloqués un certain temps à réfléchir à la façon d'initier le projet, de sorte à éviter d'être bloqués à mi-chemin. Cependant, grâce à nos recherches et à nos concertations, nous avons réussi à mettre en place une bonne architecture qui nous a permis de faciliter le développement de l'application tout au long du processus.

Un autre défi technique majeur que nous avons dû relever concernait le moteur temporel de notre simulation, géré par l'objet JavaFX Timeline. Dans nos premières implémentations, pour permettre à l'utilisateur de modifier la vitesse d'exécution de la simulation (multiplicateurs x2, x4, x8), notre approche initiale consistait à instancier une nouvelle Timeline avec un délai (Duration) plus court à chaque clic sur un bouton. Cependant, nous avons rapidement fait face à des comportements erratiques : les anciennes boucles temporelles n'étaient pas toujours détruites par le Garbage Collector, ce qui entraînait l'exécution parallèle de plusieurs Timeline. Par conséquent, la méthode `simulation.move()` était appelée de manière concurrente et anarchique, provoquant des "téléportations" d'agents et des crashes de l'interface dus à la surcharge du rafraîchissement (`layout()`).

Pour résoudre ce problème critique de désynchronisation, nous avons dû repenser la gestion de notre boucle principale en adoptant une approche par mutation d'état. Plutôt que de recréer des objets, nous avons implémenté une méthode centralisée (`updateLoopSpeed`) qui conserve une instance unique de la Timeline. Lors d'un changement de vitesse, le programme stoppe proprement la boucle actuelle (`loop.stop()`), vide la liste des événements temporels pour y injecter une nouvelle KeyFrame écrasant la précédente (`loop.getKeyFrames().setAll(...)`), avant de relancer

l'animation. Cette solution architecturale nous a permis de garantir une simulation parfaitement stable, des transitions de vitesse instantanées, et surtout d'éliminer totalement les fuites de mémoire liées aux boucles fantômes.

Conclusion

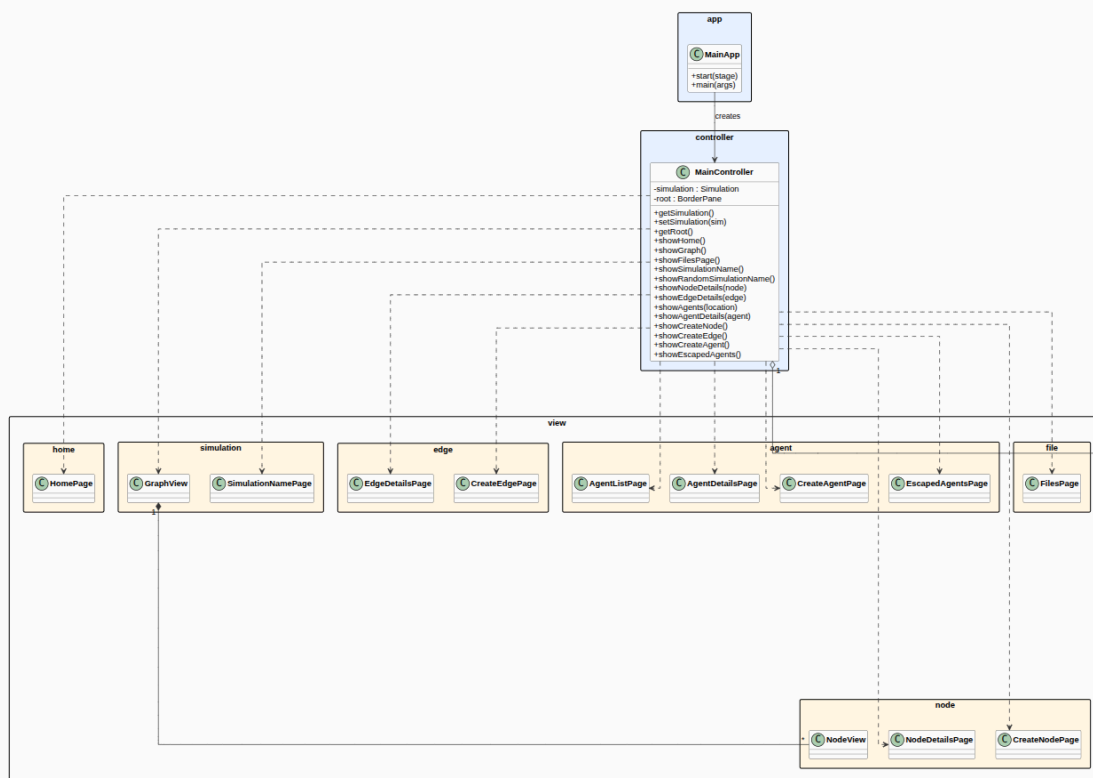
Ce projet nous a permis de découvrir une autre facette de ce type de projet qu'est le génie logiciel, en apprenant les concepts et en comprenant comment les applications fonctionnent et sont développées avec une architecture MVC (Model - View - Controller) et une pensée orientée objet (Java).

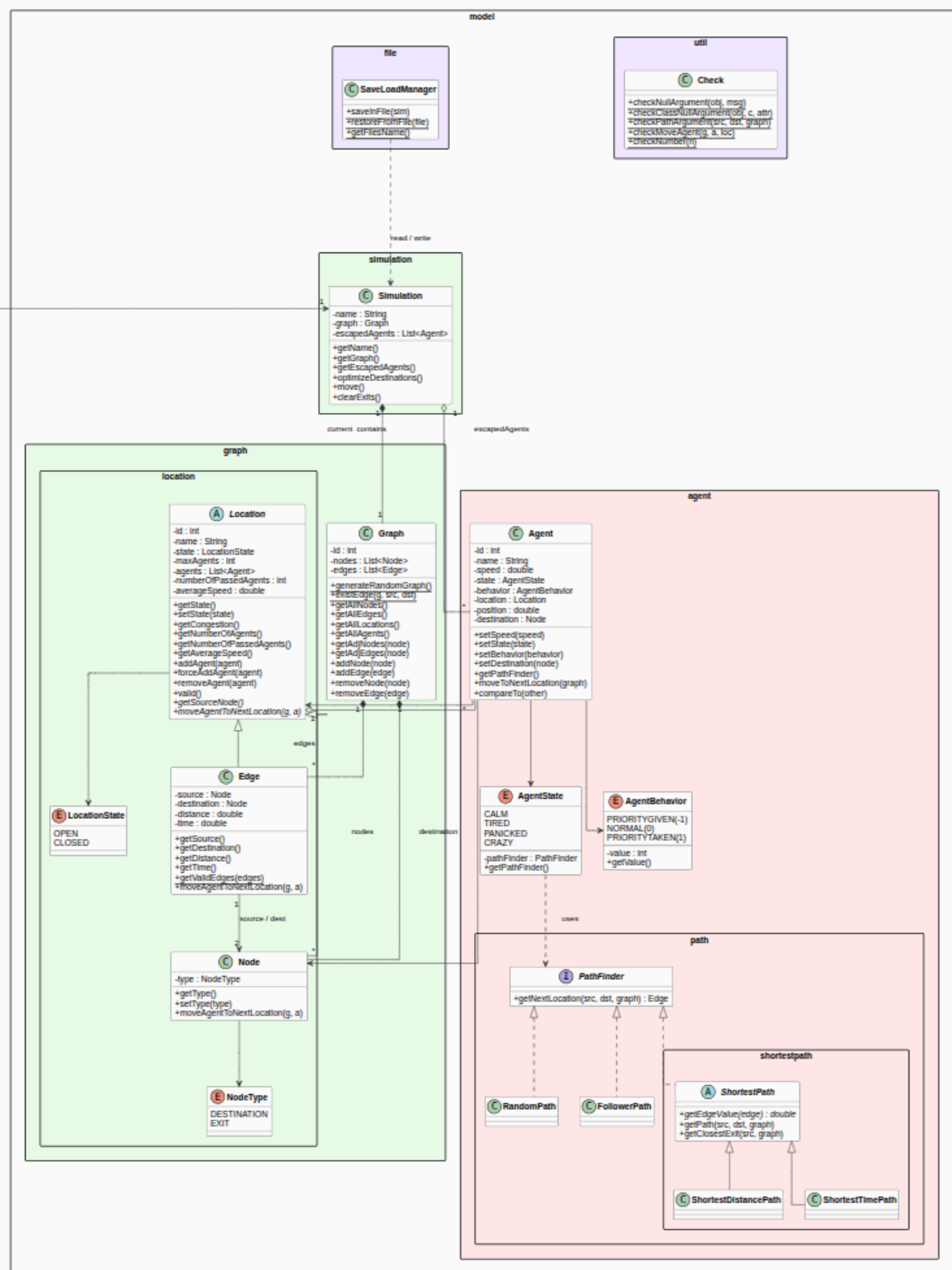
Notre tutrice, quant à elle, nous a permis de nous mettre en situation de monde professionnel en respectant plusieurs deadlines tout au long du projet, ainsi qu'en utilisant des outils très utilisés au sein des entreprises comme Jira, qui permet de faciliter l'assignation des tâches et le travail au sein des équipes, sans oublier les différents termes techniques de ce secteur que nous avons pu apprendre.

Enfin, cette application nous a permis de développer nos compétences en programmation orientée objet en premier plan, en s'appuyant sur la pensée objet lors de l'implémentation de la logique métier, en plus d'utiliser un framework (JavaFX) pour créer des interfaces graphiques modernes, permettant de visualiser de façon intuitive notre système.

Annexes

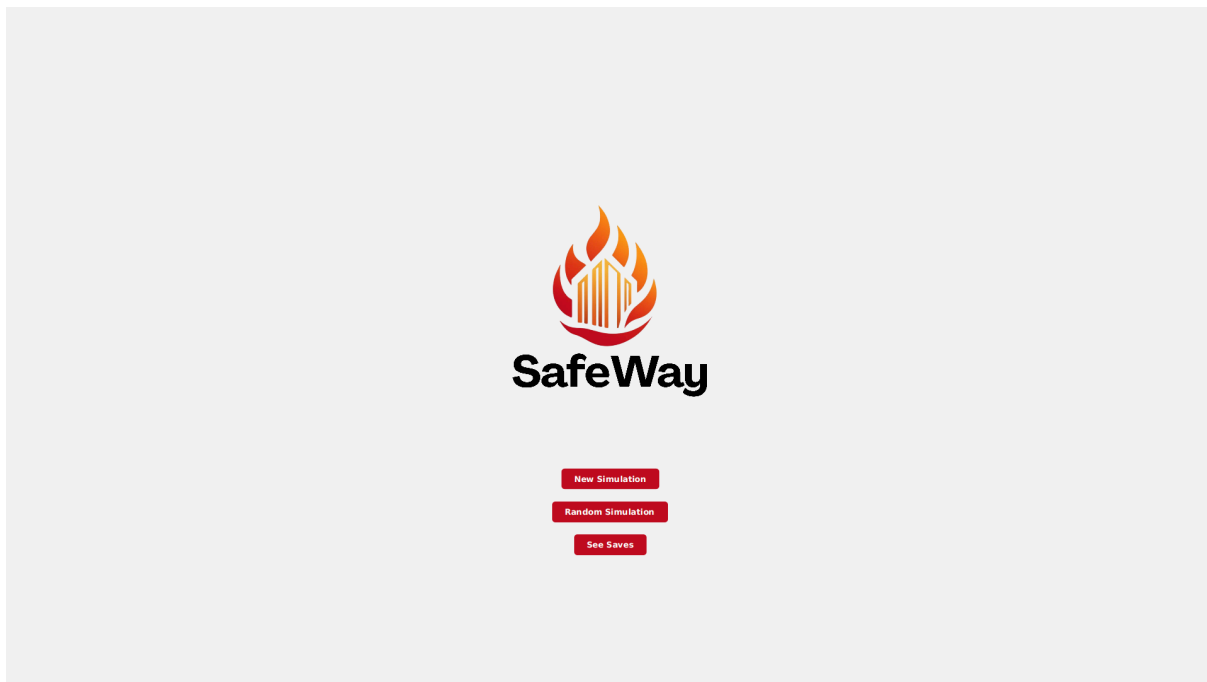
- Diagramme de classes :



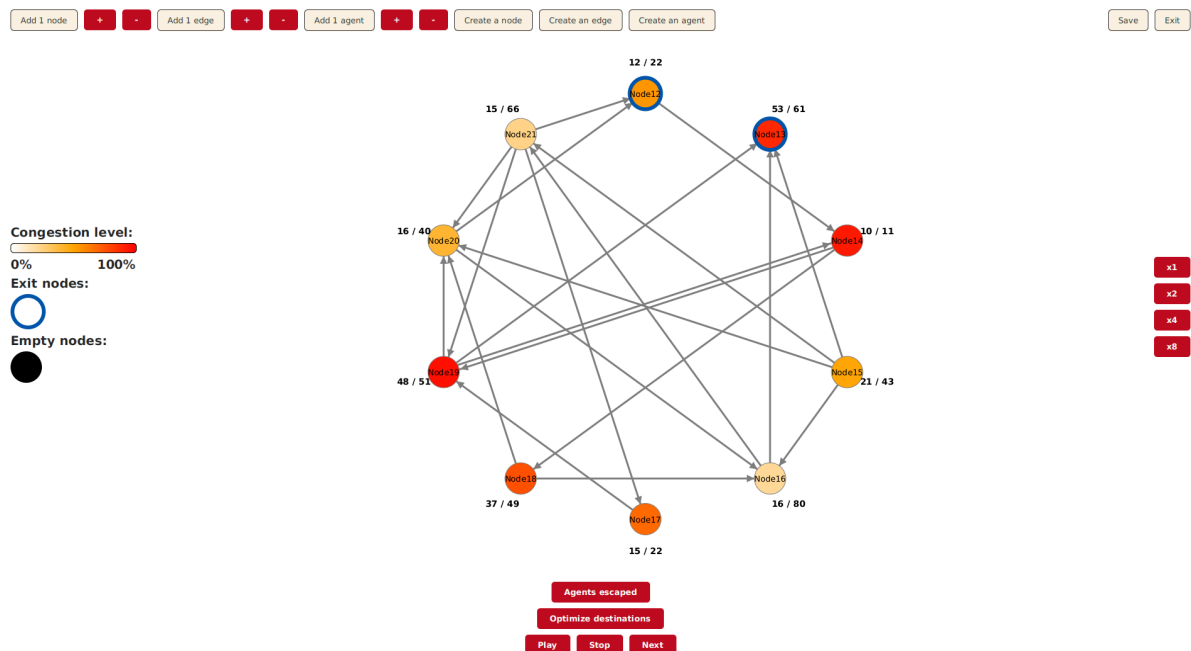


- Quelques images de l'application :

- Page d'accueil :



- Page avec vue du graphe et des différentes fonctionnalités :



- Page contenant les informations d'un noeud :

[Back to graph](#)

Node details

[Create an agent](#) [Create an edge](#) [See agents](#) [Delete node](#)

ID: 13
Name: Node13
State: CLOSED
Type: EXIT
Agents: 10 / 17

[Modify](#)

- Page de création d'un agent :

[Back](#)

Create an agent on

Start node :

Name :

Speed :

State :

Behavior :

Congestion : ☒ Tolerant to congestion

Destination :

[Create](#)

- Page de création d'une arête :

Back

Create an edge

Name :

State :

OPEN

Max agents :

20

Source :

Destination :

Distance :

> 0

Create

- Page de création d'un agent :

Back

Create an agent on

Start node :

Name :

Speed :

1000

State :

CALM

Behavior :

NORMAL

Destination :

Create

- Page affichant les agents ayant réussi l'évacuation :

